Bases de données
SQL

Déclencheurs, automatismes et autres actions compensatoires

SQL_06b
v132a
2025-04-01

Christina.Khnaisser@USherbrooke.ca
Luc.Lavoie@USherbrooke.ca

© 2018-2021, **Μῆτις** (http://info.usherbrooke.ca/llavoie)
CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/)

Plan

- Rappels
  - Les invariants (ASSERTION)
  - Les vues (VIEW)
- Les déclencheurs (TRIGGER)
- Les automatismes (TRIGGER FUNCTION)
- Perspectives
- Exemples
- Références

Rappels

- Pincipes et caractérisation
  (fonction, procédure, automatisme)
- Invariants (ASSERTION)
- Vues (VIEW)

Rappels
Principes et caractérisation : fonctions, procédures et automatismes

- Qu'ont en commun une fonction, une procédure et un automatisme ?
  - *Ce sont des abstractions paramétrables d'un traitement.*

- Qui fait quoi et comment ?
  - Une *fonction* **calcule une valeur** suite à un appel
    (sur la base de ses paramètres ou de l'état du calculateur, sans en changer l'état).
  - Une *procédure* **change l'état du calculateur** suite à un appel
    *(sur la base de ses paramètres **et** de l'état du calculateur).*
  - Un *automatisme* change l'état du calculateur **suite à un évènement**
    (sur la base de cet évènement, de ses paramètres et de l'état du calculateur).

- Un calculateur est en fait une machine à états.
  Une base de données est un cas particulier de calculateur.

Invariants
Un Exemple - Le problème

Soit la représentation suivante d'une cartothèque :

```
CREATE TABLE Inventaire (
   idArticle Article,
   idClasseur Classeur,
   ....
   PRIMARY KEY (idArticle)
  );
```

On désire limiter le nombre d'articles par classeur à 40.

Invariants
Un Exemple – Une solution

```
CREATE ASSERTION Inventaire_Max AS
 CHECK (
  NOT EXISTS (
      SELECT idClasseur
      FROM Inventaire
      GROUP BY idClasseur
      HAVING  COUNT(*) > 40
  )
 );
```

Les invariants (ASSERTION) sont des contraintes pouvant référer à une ou plusieurs tables et pouvant utiliser toute la « puissance » des expressions relationnelles.

Invariants
Syntaxe SQL ISO : CREATE ASSERTION

<assertion definition> ::=
CREATE ASSERTION <constraint name>
CHECK (<search condition>)
[<constraint characteristics>]

<constraint characteristics> ::=
<constraint check time> [[NOT] DEFERRABLE]
| [NOT] DEFERRABLE [<constraint check time>]

<constraint check time> ::=
INITIALLY { DEFERRED | IMMEDIATE }

<search condition> ::=
toute expression relationnelle dont le résultat est un booléen

MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mητις - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Le langage SQL
Les invariants en pratique

- Malheureusement, très peu de dialectes SQL sont conformes à la norme : le plus souvent le CREATE ASSERTION n'est pas offert, alors que c'est un pilier de la théorie relationnelle !

- En pratique, ces dialectes imposent aux concepteurs de « programmer » une ASSERTION (invariant) à l'aide de déclencheurs (TRIGGER) et d'automatismes (TRIGGER FUNCTION).

Les Vues (rappel)

- Voir TMR_03-Relation-virtuelles
- Voir SQL_05-Vues

Automatismes
Syntaxe SQL ISO (1/2)

```
<trigger definition> ::=
   CREATE TRIGGER <trigger name>
   <trigger action time> <trigger event>  ON <table name>
   [ REFERENCING <transition list> ]
   <triggered action>

<trigger action time> ::=
   BEFORE   | AFTER  | INSTEAD OF
<trigger event> ::=
   INSERT  | DELETE  | UPDATE [ OF <column name list> ]
```


MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrtus - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Faire la représentation graphique au tableau

Automatismes
Syntaxe SQL ISO (2/2)

<triggered action> ::=
[FOR EACH { ROW | STATEMENT }]
[WHEN (<search condition>)]
<triggered SQL statement>

<triggered SQL statement> ::=
<SQL procedure statement> | BEGIN [ATOMIC] <instructions> END

<instructions> ::=
{ <SQL procedure statement> ; ...}

...

2025-04-01 MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mητις - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Faire la représentation graphique au tableau

Automatismes
Syntaxe PostgreSQL (1/2)

```
<trigger definition> ::=
   CREATE [ OR REPLACE ] [ CONSTRAINT ] TRIGGER <trigger name>
   <trigger action time>
   [ <trigger event> [ OR ... ] ]  ON <table name>
   [ FROM <referenced table name> ]
   [ NOT DEFERRABLE | [DEFERRABLE] [INITIALLY {IMMEDIATE|DEFERRED}] ]
   [ REFERENCING <transition list> ]
   <triggered action>

<trigger action time> ::=
   BEFORE  | AFTER  | INSTEAD OF
<trigger event> ::=
   INSERT  | DELETE  | UPDATE [ OF <column name list> ]
  | TRUNCATE
```


FOR EACH ROW FOR EACH STATEMENT
This specifies whether the trigger function should be fired once for every row affected by the trigger event, or just once per SQL statement. If neither is specified, FOR EACH STATEMENT is the default. Constraint triggers can only be specified FOR EACH ROW.
Condition
A Boolean expression that determines whether the trigger function will actually be executed. If WHEN is specified, the function will only be called if the ***condition*** returns true. In FOR EACH ROW triggers, the WHEN condition can refer to columns of the old and/or new row values by writing OLD.***column_name***or NEW.***column_name*** respectively. Of course, INSERT triggers cannot refer to OLD and DELETE triggers cannot refer to NEW. INSTEAD OF triggers do not support WHEN conditions.
Currently, WHEN expressions cannot contain subqueries.
Note that for constraint triggers, evaluation of the WHEN condition is not deferred, but occurs immediately after the row update operation is performed. If the condition does not evaluate to true then the trigger is not queued for deferred execution.

When the CONSTRAINT option is specified, this command creates a *constraint trigger*. This is the same as a regular trigger except that the timing of the trigger firing can be adjusted using **SET CONSTRAINTS**. Constraint triggers must be AFTER ROW triggers on plain tables (not foreign tables). They can be fired either at the end of the statement causing the triggering event, or at the end of the containing transaction; in the latter case they are said to be *deferred*. A pending deferred-trigger firing can also be forced to happen immediately by using SET CONSTRAINTS. Constraint triggers are expected to raise an exception when the

constraints they implement are violated.

The REFERENCING option enables collection of *transition relations*, which are row sets that include all of the rows inserted, deleted, or modified by the current SQL statement. This feature lets the trigger see a global view of what the statement did, not just one row at a time. This option is only allowed for an AFTER trigger that is not a constraint trigger; also, if the trigger is an UPDATE trigger, it must not specify a ***column_name*** list. OLD TABLE may only be specified once, and only for a trigger that can fire on UPDATE or DELETE; it creates a transition relation containing the *before-images* of all rows updated or deleted by the statement. Similarly, NEW TABLE may only be specified once, and only for a trigger that can fire on UPDATE or INSERT; it creates a transition relation containing the *after-images* of all rows updated or inserted by the statement.

Automatismes
Syntaxe PostgreSQL (2/2)

```
<transition list> ::=
   { { OLD | NEW } TABLE [ AS ] transition_relation_name } [ ... ]

<triggered action> ::=
   [ FOR [ EACH ] { ROW | STATEMENT } ]
      [ WHEN ( <search condition> ) ]
         <triggered SQL statement>

<triggered SQL statement> ::=
   EXECUTE {FUNCTION|PROCEDURE} function_name ( arguments )
```


2025-04-01 MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrftcs - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

name
The name to give the new trigger. This must be distinct from the name of any other trigger for the same table. The name cannot be schema-qualified — the trigger inherits the schema of its table. For a constraint trigger, this is also the name to use when modifying the trigger's behavior using SET CONSTRAINTS.

BEFORE
AFTER
INSTEAD OF
Determines whether the function is called before, after, or instead of the event. A constraint trigger can only be specified as AFTER.

event
One of INSERT, UPDATE, DELETE, or TRUNCATE; this specifies the event that will fire the trigger. Multiple events can be specified using OR, except when transition relations are requested.
For UPDATE events, it is possible to specify a list of columns using this syntax:
UPDATE OF ***column_name1*** [, ***column_name2*** ...] The trigger will only fire if at least one of the listed columns is mentioned as a target of the UPDATE command or if one of the listed columns is a generated column that depends on a column that is the target of the UPDATE.
INSTEAD OF UPDATE events do not allow a list of columns. A column list cannot be specified when requesting transition relations, either.

table_name
The name (optionally schema-qualified) of the table, view, or foreign table the trigger is for.

referenced_table_name
The (possibly schema-qualified) name of another table referenced by the constraint. This option is used for foreign-key constraints and is not recommended for general use. This can only be specified for constraint triggers.

DEFERRABLE
NOT DEFERRABLE
INITIALLY IMMEDIATE
INITIALLY DEFERRED
The default timing of the trigger. See the **CREATE TABLE** documentation for details of these constraint options. This can only be specified for constraint triggers.

REFERENCING
This keyword immediately precedes the declaration of one or two relation names that provide access to the transition relations of the triggering statement.

OLD TABLE
NEW TABLE
This clause indicates whether the following relation name is for the before-image transition relation or the after-image transition relation.

transition_relation_name
The (unqualified) name to be used within the trigger for this transition relation.

FOR EACH ROW
FOR EACH STATEMENT
This specifies whether the trigger function should be fired once for every row affected by the trigger event, or just once per SQL statement. If neither is specified, FOR EACH STATEMENT is the default. Constraint triggers can only be specified FOR EACH ROW.

condition
A Boolean expression that determines whether the trigger function will actually be executed. If WHEN is specified, the function will only be called if the ***condition*** returns true. In FOR EACH ROW triggers, the WHEN condition can refer to columns of the old and/or new row values by writing OLD.***column_name*** or NEW.***column_name*** respectively. Of course, INSERT triggers cannot refer to OLD and DELETE triggers cannot refer to NEW. INSTEAD OF triggers do not support WHEN conditions.
Currently, WHEN expressions cannot contain subqueries.
Note that for constraint triggers, evaluation of the WHEN condition is not deferred, but occurs immediately after the row update operation is performed. If the condition does not evaluate to true then the trigger is not queued for deferred execution.

function_name
A user-supplied function that is declared as taking no arguments and returning type trigger, which is executed when the trigger fires.
In the syntax of CREATE TRIGGER, the keywords FUNCTION and PROCEDURE are equivalent, but the referenced function must in any case be a function, not a procedure. The use of the keyword PROCEDURE here is historical and deprecated.

arguments
An optional comma-separated list of arguments to be provided to the function when the trigger is executed. The arguments are literal string constants. Simple names and numeric constants can be written here, too, but they will all be converted to strings. Please check the description of the implementation language of the trigger function to find out how these arguments can be accessed within the function; it might be different from normal function arguments.

Automatismes (trigger function)
Syntaxe – simplifiée (PostgreSQL)

trigger_function_def ::=
CREATE [OR REPLACE] FUNCTION *name* ([*argument* [, ...]])
RETURNS *TRIGGER*
{ *other_fun_qual* } ...
LANGUAGE *lang_name*
AS *body*

argument ::=
[*arg_mode*] [*arg_name*] *arg_type* [DEFAULT *default_expr*]

arg_mode ::=
IN | VARIADIC

2025-04-01 MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mr†ɪ‡ - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

En pratique les "trigger function" doivent être écrites en plpgsql.

other_fun_qual ::=
WINDOW
| TRANSFORM { FOR TYPE ***type_name*** } [, ...]
~~| IMMUTABLE | STABLE | VOLATILE | [NOT] LEAKPROOF~~
| CALLED ON NULL INPUT | RETURNS NULL ON NULL INPUT | STRICT
| [EXTERNAL] SECURITY INVOKER | [EXTERNAL] SECURITY DEFINER
| COST ***execution_cost***
| ROWS ***result_rows***
| SET ***configuration_parameter*** { TO ***value*** | = ***value*** | FROM CURRENT }
| AS '***obj_file***', '***link_symbol***'

arg_mode

En cas d'omission, le mode est IN.

Une fonction ne devrait utiliser que des arguments IN ou VARIADIC;

les modes OUT et INOUT ne sont pas permis.

DEFAULT

En PostgreSQL, peut être remplacé par le symbole **=**.

Automatismes (trigger function)
Syntaxe (PostgreSQL)

- Accès à la valeur de l'objet de référence (ROW ou TABLE)
  - soit avant l'évènement (OLD)
  - soit après l'évènement (NEW)
- Selon la syntaxe suivante

```
<reference> ::=
   OLD [ ROW ]
 | NEW [ ROW ]
 | OLD TABLE
 | NEW TABLE
```

Un Exemple
La solution par le biais d'un automatisme de TABLE

Il faut définir un « TRIGGER » et...

```
CREATE TRIGGER Inventaire_Max
  AFTER INSERT OR UPDATE
  ON Inventaire
  EXECUTE PROCEDURE
    Inventaire_lim () ;
```

une « TRIGGER FUNCTION »

```
CREATE FUNCTION Inventaire_lim ()
  RETURNS TRIGGER
  LANGUAGE plpgsql AS $$
    BEGIN
      IF NOT EXISTS (
          SELECT idClasseur
          FROM Inventaire
         GROUP BY idClasseur
         HAVING  COUNT(*) > 40 ) THEN
        RETURN NEW; -- on peut procéder
      ELSE
        RAISE EXCEPTION 'Trop de cartes';
        RETURN NULL; -- Rien à insérer
       END IF;
    END
$$
```


2025-04-01
MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mr Tuc - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Pourquoi pas AFTER INSERT OR UPDATE *OR DELETE ?*

Un Exemple
La solution par le biais d'un automatisme de TABLE … optimisée

Il faut définir un « TRIGGER » et…

```
CREATE TRIGGER Inventaire_Max
  AFTER INSERT
  ON Inventaire
  EXECUTE PROCEDURE
    Inventaire_lim () ;

CREATE TRIGGER Inventaire_Max
  AFTER UPDATE OF idClasseur
  ON Inventaire
  EXECUTE PROCEDURE
    Inventaire_lim () ;
```

une « TRIGGER FUNCTION »

```
CREATE FUNCTION Inventaire_lim ()
  RETURNS TRIGGER
LANGUAGE plpgsql AS $$
    BEGIN
        IF NOT EXISTS (
              SELECT idClasseur
              FROM Inventaire
             GROUP BY idClasseur
             HAVING  COUNT(*) > 40 ) THEN
          RETURN NEW; -- on peut procéder
        ELSE
           RAISE EXCEPTION 'Trop de cartes';
           RETURN NULL; -- Rien à insérer
          END IF;
    END
$$
```

Un Exemple
La solution par le biais d'un automatisme de ROW

Il faut définir un « TRIGGER » et...

```
CREATE TRIGGER Inventaire_Max
 AFTER
  INSERT OR  UPDATE OF idClasseur
 ON Inventaire
 FOR EACH ROW
  EXECUTE PROCEDURE
     Inventaire_lim () ;
```

une « TRIGGER FUNCTION »

```
CREATE FUNCTION Inventaire_lim ()
 RETURNS TRIGGER
LANGUAGE  plpgsql  AS $$
   BEGIN
      IF (SELECT COUNT(idArticle)
            FROM Inventaire
            WHERE (idClasseur=NEW.idClasseur)) <=
      40 THEN
        RETURN NEW; -- on peut procéder
      ELSE
        RAISE EXCEPTION 'Trop de cartes';
        RETURN NULL; -- rien à insérer
      END IF;
   END
$$
```


MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrtus - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Un Exemple
La solution à l'aide d'un automatisme de ROW

Il faut définir un « TRIGGER » et...

```
CREATE TRIGGER Inventaire_Max
 AFTER
  INSERT OR  UPDATE OF idClasseur
 ON Inventaire
 FOR EACH ROW
  EXECUTE PROCEDURE
     Inventaire_lim () ;
```

C'est toutefois une « mauvaise » idée, pourquoi ?

une « TRIGGER FUNCTION »

```
CREATE FUNCTION Inventaire_lim ()
 RETURNS TRIGGER
 LANGUAGE  plpgsql  AS $$
   BEGIN
      IF (SELECT COUNT(idArticle)
            FROM Inventaire
            WHERE (idClasseur=NEW.idClasseur)) <=
      40 THEN
        RETURN NEW; -- on peut procéder
      ELSE
        RAISE EXCEPTION 'Trop de cartes';
        RETURN NULL; -- rien à insérer
      END IF;
   END
$$
```

Automatismes
Quels évènements avec quoi?

Le tableau suivant récapitule les types d'automatismes pouvant être utilisés sur les tables et les vues :

When	Event	Row-level	Statement-level
BEFORE	INSERT UPDATE DELETE	Tables et *tables distantes*	Tables, **vues** et *tables distantes*
	TRUNCATE	—	Tables
AFTER	INSERT UPDATE DELETE	Tables et *tables distantes*	Tables, **vues** et *tables distantes*
	TRUNCATE	—	Tables
INSTEAD OF	INSERT UPDATE DELETE	**Vues**	—
	TRUNCATE	—	—

Source : https://docs.postgresql.fr/current/sql-createtrigger.html

2025-04-01 MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrtus - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

views : on peut donc programmer les opérations insert, update et delete sur les vues !
foreign tables : on peut donc « lier » deux bases de données !!!

Exemple de données

Étudiant

matricule	nom	adresse
15113150	Paul	>ᕕᖅᓂᑐᖅ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

TypeÉvaluation

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Activité

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

Résultat

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20133	80
15112354	FI	IFT187	20123	78
15113150	TP	IFT159	20133	75
15112354	FI	GMQ103	20123	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

2025-04-01 MCED_SQL_06b : Invriants et automatismes (v132a) © 2018-2025, Mᖅᑭᖅ - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

On remarque l'omission de la dénotation des types des attributs.
C'est fréquemment le cas dans les représentations graphiques, afin de les alléger.
Cela ne porte pas à conséquence dans la mesure où une définition textuelle les accompagne, ce qui est le cas ici.

Nous verrons bientôt d'autres représentations graphiques plus complètes.

Note : >ᕕᖅᓂᑐᖅ se prononce (approximativement) Puvirnituq

Évaluation
Exemple 1 (v1)

Un étudiant ne peut pas s'inscrire à plus de 5 cours au même trimestre

```
CREATE FUNCTION Cours_lim
  RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
 IF NOT EXISTS (
   SELECT * FROM Resultat
   GROUP BY matricule, trimestre
   HAVING COUNT(DISTINCT activite) > 5 )
 THEN
   RETURN NEW; -- on peut procéder
 ELSE
   RAISE EXCEPTION 'Trop de cours';
   RETURN NULL; -- rien à insérer
 END IF;
END
$$

CREATE TRIGGER Cours_dec_lim
  AFTER INSERT OR UPDATE ON Resultat
  EXECUTE PROCEDURE Cours_lim();
```

 © 2018-2025, Mr†ıς - CC BY-NC-SA 4.0

Évaluation
Exemple 1 (v2)

Un étudiant ne peut pas s'inscrire à plus de 5 cours au même trimestre

```
CREATE TRIGGER Cours_dec_lim
  AFTER
    INSERT
  OR
    UPDATE OF matricule, trimestre, activite
  ON Resultat
  EXECUTE PROCEDURE Cours_lim();
```


MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mr Tuc - CC BY-NC-SA 4.0

Évaluation – Exemple 2 (mise en situation)

- Ce qu'on a :

```
CREATE VIEW Inscription AS
  SELECT DISTINCT matricule, nom, adresse, activite, trimestre
  FROM Resultat JOIN Etudiant USING (matricule) ;
```

- Ce qu'on veut faire :

```
INSERT INTO Inscription
         (matricule,  nom,    adresse, activite, trimestre)
  VALUES ('18773250', 'Adam', 'Laval', 'IGE401', '20133') ;
```

- Le moyen qu'on choisit :

```
CREATE TRIGGER Inscription_ins_dec
   INSTEAD OF INSERT ON Inscription
   FOR EACH ROW EXECUTE PROCEDURE Inscription_ins () ;
```


MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mr Tuc - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Évaluation – Exemple 2 (v1)

```
CREATE OR REPLACE FUNCTION Inscription_ins ()
  RETURNS TRIGGER
  LANGUAGE plpgsql AS
  $$
  BEGIN
    INSERT INTO Etudiant (matricule, nom, adresse)
      VALUES (NEW.matricule, NEW.nom, NEW.adresse);
    INSERT INTO Resultat (matricule, te, activite, trimestre, note)
      VALUES (NEW.matricule, 'TP', NEW.activite, NEW.trimestre, 0);
    RETURN NEW;
  END;
  $$;
```


MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrtus - CC BY-NC SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Évaluation – Exemple 2 (v2)

```
CREATE OR REPLACE FUNCTION Inscription_ins()
  RETURNS TRIGGER
  LANGUAGE plpgsql AS
  $$
  BEGIN
    IF EXISTS(SELECT * FROM Etudiant WHERE matricule = NEW.matricule) THEN
      IF EXISTS (
          SELECT * FROM  Etudiant
          WHERE matricule = NEW.matricule AND
            (nom <> NEW.nom OR adresse <> NEW.adresse)
        ) THEN
        RAISE EXCEPTION 'Incohérence de nom ou d''adresse' ;
        RETURN NULL ;
      ELSE
        -- ne rien faire
      END IF ;
    ELSE
      INSERT INTO Etudiant (matricule, nom, adresse)
        VALUES (NEW.matricule, NEW.nom, NEW.adresse);
    END IF;
    INSERT INTO Resultat (matricule, te, activite, trimestre, note)
      VALUES (NEW.matricule, 'TP', NEW.activite, NEW.trimestre, 0);
    RETURN NEW;
  END;
  $$
```


2025-04-01
MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrtus - CC BY-NC SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

faire des variantes avec une admission si nécessaire et une correction d'adresse (et de nom) si nécessaire

Évaluation – Exemple 2 (v3)

```
CREATE OR REPLACE FUNCTION Inscription_ins()
  RETURNS TRIGGER
  LANGUAGE plpgsql AS
  $$
  BEGIN
    IF EXISTS (
      SELECT * FROM  Etudiant
      WHERE matricule = NEW.matricule AND
            (nom <> NEW.nom OR adresse <> NEW.adresse)
      )
    THEN
        RAISE EXCEPTION 'Incohérence de nom ou d''adresse' ;
        RETURN NULL ;
    ELSE
      INSERT INTO Etudiant (matricule, nom, adresse)
        VALUES (NEW.matricule, NEW.nom, NEW.adresse)
        ON CONFLICT DO NOTHING ;
    END IF;
    INSERT INTO Resultat (matricule, te, activite, trimestre, note)
      VALUES (NEW.matricule, 'TP', NEW.activite, NEW.trimestre, 0);
    RETURN NEW;
  END;
  $$
```


MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mrtus - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

faire des variantes avec une admission si nécessaire et une correction d'adresse (et de nom) si nécessaire

Automatismes
Autres exemples PostgreSQL

- Exemples
  - https://docs.postgresql.fr/current/sql-createtrigger.html

MCED_SQL_06b : Invariants et automatismes (v132a) © 2018-2025, Mr Tuc - CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Faire la représentation graphique au tableau

Compléments

- Pour des exemples plus complexes, voir :
  - https://www.postgresql.org/docs/current/plpgsql-trigger.html
- Pour d'autres types de trigger (e.a.: «event trigger»), voir :
  - https://www.postgresql.org/docs/current/sql-createeventtrigger.html
  - https://www.postgresql.org/docs/current/event-triggers.html

Automatismes
Perspectives

- L'absence du CREATE ASSERTION dans la majorité des dialectes SQL est généralement « motivée » par le cout induit par leur vérification systématique. Quand on y regarde de plus près, force est de constater que le SGBD est toujours capable de faire aussi bien, sinon mieux, que le programmeur lorsque vient le temps de traduire l'assertion en automatisme. En ce qui concerne le déclencheur, cela est aussi le plus souvent le cas, surtout si le compilateur s'inspire des techniques développées pour les langages fonctionnels.
- Il appartient donc aux utilisateurs de faire des pressions sur les éditeurs pour que ceux-ci passent à l'action!
- Les automatismes et les déclencheurs demeurent toutefois pertinents dans d'autres situations.

Références

- Elmasri et Navathe (4e éd.), chapitre 7
- Elmasri et Navathe (6e éd.), chapitre 4
- [Loney2008]
  Loney, Kevin ;
  Oracle Database 11g: The Complete Reference.
  Oracle Press/McGraw-Hill/Osborne, 2008.
  ISBN 978-0071598750.
- [Date2012]
  Date, Chris J. ;
  SQL and Relational Theory: How to Write Accurate SQL Code.
  2nd edition, O'Reilly, 2012.
  ISBN 978-1-449-31640-2.
- Le site d'Oracle (en anglais)
  - http://www.oracle.com/pls/db10g/portal.portal_demo3?selected=5
  - http://docs.oracle.com/cd/B19306_01/server.102/b14200/toc.htm
- Le site de PostgreSQL (en français)
  - http://docs.postgresqlfr.org